

Individual Assignment

Customer Churn Prediction:

*Logistic Regression, Neural Networks and CatBoost with SHAP
Interpretability*

Program Name: MSc Artificial Intelligence

Module Name: Neural Nets and Deep Learning

Student Full Name: Charalampos Lazaridis Patsalias

Contents

1	Data Exploration and Preprocessing	1
1.1	Description and cleaning of the dataset	1
1.2	Data preprocessing and transformation	2
1.2.1	Handling missing values and cleaning	2
1.2.2	Encoding categorical variables	3
1.2.3	Feature selection and handling multicollinearity	4
1.2.4	Train-test split and data normalisation	8
2	Development of the Logistic Regression Model	9
2.1	Optimisation strategy: GridSearchCV	9
2.2	Results and evaluation	10
2.3	Advantages and limitations of the model	13
3	Development and Optimisation of the Neural Network	13
3.1	Architecture and design strategy	13
3.2	Hyperparameter optimisation with Optuna	15
3.2.1	Definition of the search space	15
3.2.2	Evaluation strategy and pruning	15
3.2.3	Optimisation results	16
3.3	Handling class imbalance	17
3.4	Evaluation of the final Neural Network	18
4	Advanced Modelling with CatBoost	21
4.1	Optimisation and training	21
4.2	Results and comparison	22
5	Comparative Analysis and Final Model Selection	23
5.1	Performance comparison	23
5.1.1	Predictive performance	23
5.1.2	Computational cost and training times	24
5.1.3	Final selection and critical evaluation	25
6	Model Interpretability with SHAP	25
6.1	Global feature importance	25
6.2	Direction and nature of the effect	27
6.3	Local interpretation	27
7	Conclusions and Future Directions	29
8	Bibliography	30

A	Explanations of Methods and Algorithms	31
A.1	Explanation of the Variance Inflation Factor	31
A.2	The ReduceLROnPlateau mechanism	31
A.3	The CatBoost algorithm (Categorical Boosting)	31
A.4	SHAP theory (SHapley Additive exPlanations)	32
B	Code	33

List of Figures

1	Box plots of the standardised numeric variables (tenure, MonthlyCharges, TotalCharges) split by churn.	1
2	Distribution of the target variable: 73.5% of customers stay (No) versus 26.5% who churn (Yes).	2
3	Code: dropping the customerID column, a unique identifier with no predictive value.	2
4	Code: counting NaN and empty-string values per column to locate missing data.	3
5	Code: converting TotalCharges to numeric and removing the 11 rows with missing values.	3
6	Code: binary encoding of two-category variables to 0/1 values.	4
7	Code: one-hot encoding of multi-category variables with drop_first=True.	4
8	Code: computing the Pearson and Spearman correlation of every feature with the target.	5
9	Top 20 features by Pearson (left) and Spearman (right) correlation with churn. The two rankings are nearly identical, a first sign that the relationships are mostly linear.	5
10	Code: detecting feature pairs with correlation above 0.99 for removal.	6
11	Code: the iterative VIF loop; among correlated features it keeps the one most correlated with churn.	7
12	Correlation heatmap of the 20 final features after VIF; the maximum absolute correlation is 0.56, well below the 0.8 target.	8
13	Code: stratified 80/20 train-test split preserving the 73.4/26.6 class ratio.	8
14	Code: StandardScaler fitted on the training set only, then applied to both sets.	9
15	Code: the GridSearchCV setup for logistic regression (24 hyperparameter combinations, 5-fold stratified CV, F1 scoring).	10
16	Code: computing the evaluation metrics and confusion matrix on the test set.	11
17	Test-set evaluation of the tuned logistic regression: confusion matrix, ROC curve (AUC 0.8287), the main metrics (F1 0.6015, recall 0.7727), and the predicted probability distribution per class.	11
18	The 20 coefficients of the tuned logistic regression. Fiber optic, electronic check and paperless billing push towards churn; tenure and one/two-year contracts protect against it.	12

19	Final summary of the F1-optimised logistic regression: configuration, test performance, business impact (289 of 374 churners caught, 298 false alarms) and training cost (120 fits in 8.15 s).	12
20	Code: the dynamic Keras model builder; input layer and hidden dense layers with L2 regularization.	14
21	Code: batch normalisation, ReLU and dropout inside each hidden layer, and the two-neuron softmax output.	14
22	Code: the Optuna search space and the training call with class weights, early stopping and ReduceLROnPlateau.	15
23	Code: the Optuna study with the TPE sampler and the Median Pruner for early termination of weak trials.	16
24	Optuna optimisation history: cross-validated F1 per trial, with the best value (0.6328) marked by the dashed line.	16
25	Code: strategy 1 for class imbalance, training with class weights.	17
26	Code: strategy 2 for class imbalance, SMOTE resampling of the training set to a 1:1 ratio.	18
27	Learning curves of the final neural network: training and validation loss and F1 converge smoothly, with no overfitting gap.	19
28	Confusion matrix of the neural network on the test set: 302 of 374 churners detected (recall 80.75%).	19
29	Test metrics of the neural network: ROC-AUC 0.8351, F1 0.6189, recall 0.8075, precision 0.5017, accuracy 0.7356.	20
30	F1 comparison of the tuned logistic regression (0.6015) and the neural network (0.6189).	20
31	Code: the CatBoost model builder used inside the Optuna objective (GPU training, F1 eval metric, early stopping).	21
32	Code: the CatBoost hyperparameter search space and the Optuna study with 5-fold stratified CV.	22
33	Test-set evaluation of the tuned CatBoost model: confusion matrix, ROC curve (AUC 0.8360), the main metrics (F1 0.6247, recall 0.7701), and the predicted probability distribution per class.	23
34	F1-score comparison across the four setups: CatBoost 0.6247, neural network 0.6189, logistic regression 0.6015, neural network with SMOTE 0.5838.	24
35	Full metric comparison of the four setups on the test set (accuracy, precision, recall, F1, ROC-AUC).	24
36	Code: computing SHAP values with the TreeExplainer and producing the beeswarm and bar summary plots.	26
37	Global SHAP feature importance (mean absolute SHAP value): Contract, MonthlyCharges and tenure dominate the churn decision.	26
38	SHAP summary (beeswarm) plot for the CatBoost model. Each dot is a customer; red means a high feature value, blue a low one, grey a categorical feature.	27

39	Code: computing SHAP values for a single customer and building the waterfall plot.	28
40	Waterfall plot for one customer: starting from the base value $E[f(X)] = -0.168$, each feature pushes the prediction up (red) or down (blue) to $f(x) = -2.296$	28
41	Confusion matrix of the final CatBoost model: 288 of 374 churners caught, 260 false alarms.	29

List of Tables

1	Best hyperparameters for Logistic Regression.	10
2	Best hyperparameters for the Neural Network.	17

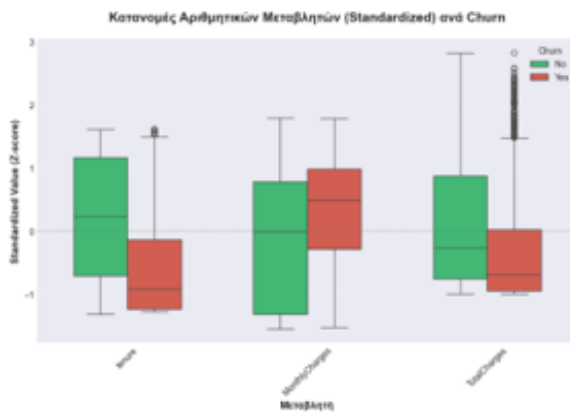
1 Data Exploration and Preprocessing

1.1 Description and cleaning of the dataset

The Telco Customer Churn dataset (Kaggle, n.d.) includes 7,043 records, 20 features and 1 target variable. The features are divided into four categories:

- demographic (gender, SeniorCitizen, Partner, Dependents),
- service usage (PhoneService, InternetService, OnlineSecurity, etc.),
- financial (MonthlyCharges, TotalCharges),
- contract (Contract, PaymentMethod, PaperlessBilling).

Most features were categorical (2 or 3 categories). Tenure and TotalCharges were also categorical at first, with TotalCharges containing 11 missing values, but both were converted to numeric. The box plot below depicts the distributions of the numeric variables.



Εικόνα 1. Θηκόγραμμα αριθμητικών μεταβλητών χωρισμένα ως προς το Churn. Το θηκόγραμμα αποκαλύπτει ότι οι πελάτες με μικρή διάρκεια παραμονής είναι πιο πιθανό να διακόψουν τη συνεργασία τους. Αυτό γίνεται αντιληπτό από το γεγονός ότι η κατανομή του χρόνου παραμονής για όσους φεύγουν είναι έντονα θετικά ασύμμετρη (δεξιά). Αντιθέτως, φαίνεται πως αποχωρούν και πελάτες με μεγάλες μηνιαίες χρεώσεις, καθώς η αντίστοιχη κατανομή παρουσιάζει κάποια αρνητική ασυμμετρία (αριστερά).

Figure 1: Box plots of the standardised numeric variables (tenure, MonthlyCharges, TotalCharges) split by churn.

The target variable indicates whether the customer churned or remained with the service. Its distribution is imbalanced, with approximately 26–27% churn and 73–74% retention, which creates the class imbalance problem.

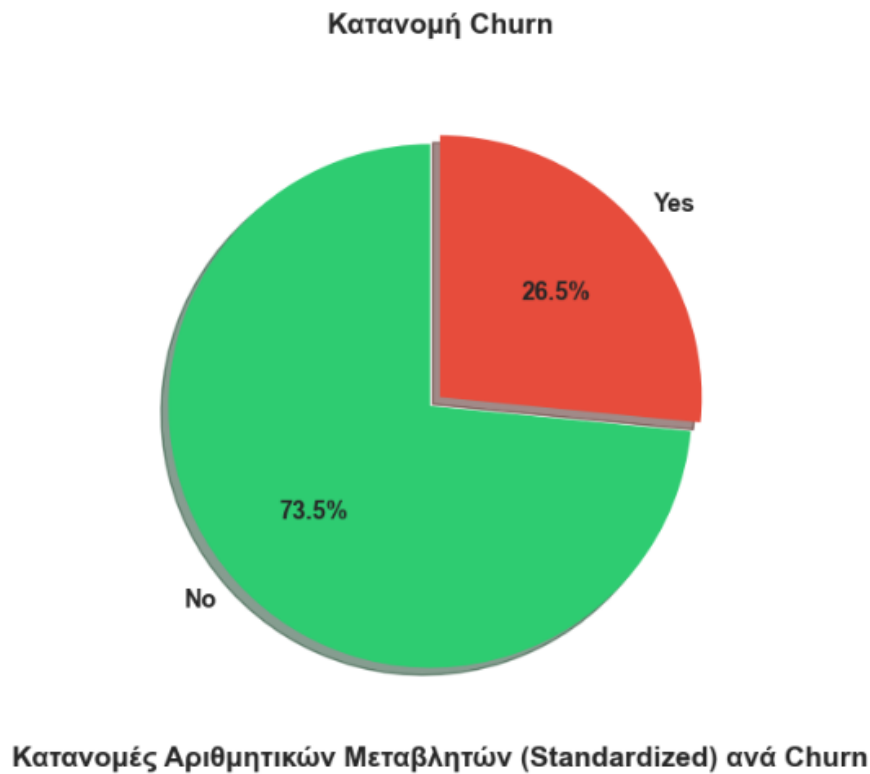


Figure 2: Distribution of the target variable: 73.5% of customers stay (No) versus 26.5% who churn (Yes).

1.2 Data preprocessing and transformation

The preprocessing procedure is critical for converting the raw data into a form suitable for training Machine Learning and Neural Network algorithms. The following steps were followed:

1.2.1 Handling missing values and cleaning

First, the customerID feature was removed since it is a unique identifier with no predictive value. As for the missing values of TotalCharges, given that they constituted a negligible percentage of the total ($<0.2\%$), they were deleted, so that the integrity of the data is preserved without introducing artificial noise. The corresponding code follows:

```
# Αφαιρούμε τις στήλες χωρίς προγνωστική αξία
drop_cols = ['customerID']
# Αν όλες οι στήλες είναι μέσα στο dataframe, τις αφαιρούμε όλες μαζί
if all(c in df_prep.columns for c in drop_cols):
    df_prep.drop(columns=drop_cols, inplace=True)
print(f"\nΑφαιρέθηκαν στήλες: {'', '.join(drop_cols)}")
```

Figure 3: Code: dropping the customerID column, a unique identifier with no predictive value.

```

# 1. Έλεγχος για NaN τιμές.
missing_values = df.isnull().sum()
# 2. Έλεγχος για empty string
# Πρώτα δημιουργούμε ένα empty pandas series
empty_strings = pd.Series(0, index=df.columns)
# Για κάθε κατηγορική μεταβλητή
for col in df.select_dtypes(include=['object']).columns:
    # Αν βρεθούνε empty string ή white space τα προσθέτουμε στο empty_strings
    empty_strings[col] = (df[col].str.strip() == '').sum()
# 3. Στο τέλος υπολογίζουμε τις συνολικές ελλιπής τιμές είτε έρχονται από NaN είτε
από empty strings.
total_missing = missing_values + empty_strings
# Κρατάμε το ποσοστό επι του συνολικού dataset για τα missing values
missing_percent = (total_missing / len(df)) * 100
# Δημιουργία dataframe για εμφάνιση πληροφοριών σχετικά με τις ελλιπής τιμές.
missing_df = pd.DataFrame({
    'Στήλη': total_missing.index,
    'NaN Τιμές': missing_values.values,
    'Empty Strings': empty_strings.values,
    'Σύνολο Ελλιπών': total_missing.values,
    'Ποσοστό (%)': missing_percent.values
})

```

Figure 4: Code: counting NaN and empty-string values per column to locate missing data.

```

# Η μόνη στήλη με ελλιπής τιμές είναι το total_charges. Για να τις αφαιρέσουμε
πρώτα την μετατρέπουμε σε αριθμητική ώστε να μετατραπούν τα empty strings σε NaN
df_prep['TotalCharges'] = pd.to_numeric(df_prep['TotalCharges'], errors='coerce')
# Αφαίρεση των ελλιπών τιμών
df_prep = df_prep.dropna().reset_index(drop=True)

```

Figure 5: Code: converting TotalCharges to numeric and removing the 11 rows with missing values.

1.2.2 Encoding categorical variables

To convert the categorical data into numeric form, two strategies were applied:

- Binary Encoding: Variables with two categories (e.g. Gender, Partner, as well as the target variable Churn) were mapped directly to values 0 and 1.
- One-Hot Encoding: For variables with multiple categories, dummy variables were created. We used the parameter `drop_first=True`, which removes one of the resulting columns to avoid multicollinearity (Hancock & Khoshgoftaar, 2020).

```

# Εντοπισμός κατηγορικών στηλών (εκτός Churn που ήδη μετατράπηκε)
cat_cols = df_prep.select_dtypes(include=['object', 'category']).columns.tolist()

# 4a) BINARY ENCODING (2 κατηγορίες)
# Βρίσκουμε τις κατηγορικές μεταβλητές με 2 κατηγορίες
binary_cols = [c for c in cat_cols if df_prep[c].nunique(dropna=True) == 2]

# Μετατρέπουμε κάθε μία από αυτές σε 0-1
for col in binary_cols:
    vals = sorted(df_prep[col].dropna().unique())
    # Έξυπνο mapping: Yes/No, Male/Female
    if set(vals) == {'No', 'Yes'}:
        mapping = {'No': 0, 'Yes': 1}
    elif set(vals) == {'Female', 'Male'}:
        mapping = {'Female': 0, 'Male': 1}
    else:
        mapping = {vals[0]: 0, vals[1]: 1}

    df_prep[col] = df_prep[col].map(mapping).astype('int8')
    print(f"      • {col}: {mapping}")

# 4b) ONE-HOT ENCODING (3-4 κατηγορίες)
# Βρίσκουμε τις κατηγορικές μεταβλητές με 3-4 κατηγορίες
onehot_cols = [c for c in cat_cols if 3 <= df_prep[c].nunique(dropna=True) <= 4]
# Αν βρεθούν τέτοιες μεταβλητές τότε αντικαθιστούμε αυτές τις στήλες του dataframe
# με τις αντίστοιχες one-hot-encoding

```

Figure 6: Code: binary encoding of two-category variables to 0/1 values.

```

if onehot_cols:
    print(f"      • Στήλες: {', '.join(onehot_cols)}")
    df_prep = pd.get_dummies(df_prep, columns=onehot_cols, drop_first=True,
dtype='int8')
    new_cols = [c for c in df_prep.columns if any(oc in c for oc in onehot_cols)]
    print(f"      • Δημιουργήθηκαν {len(new_cols)} νέες στήλες")

```

Figure 7: Code: one-hot encoding of multi-category variables with drop_first=True.

1.2.3 Feature selection and handling multicollinearity

The selection of the appropriate features was carried out in three stages, aiming to remove redundant information, strengthen the stability of the models, and reduce the required computational power.

Feature-Target correlation analysis (Pearson vs Spearman): First, the relationship of each feature with the target variable was examined using the Pearson and Spearman coefficients (James et al., 2013), which highlight linear or generally monotonic relationships respectively.

```
# Δημιουργία προσωρινού dataframe με features + target
df_analysis = X.copy()
df_analysis['Churn'] = y.values

# Υπολογισμός και των δύο τύπων συσχέτισης
pearson_corr =
df_analysis.corr(method='pearson')['Churn'].drop('Churn').sort_values(ascending=False)
spearman_corr =
df_analysis.corr(method='spearman')['Churn'].drop('Churn').sort_values(ascending=False)

# Δημιουργία συνολικού πίνακα
correlation_table = pd.DataFrame({
'Feature': pearson_corr.index,
'Pearson Correlation': pearson_corr.values,
'Spearman Correlation':
spearman_corr.reindex(pearson_corr.index).values,
'Διαφορά |Pearson - Spearman|': np.abs(pearson_corr.values -
spearman_corr.reindex(pearson_corr.index).values)
})
```

Figure 8: Code: computing the Pearson and Spearman correlation of every feature with the target.

The comparison of the two indices offers a first indication of the nature of the problem: a small deviation between them suggests linear relationships favouring Logistic Regression, while a large deviation indicates non-linear relationships where Neural Networks excel.

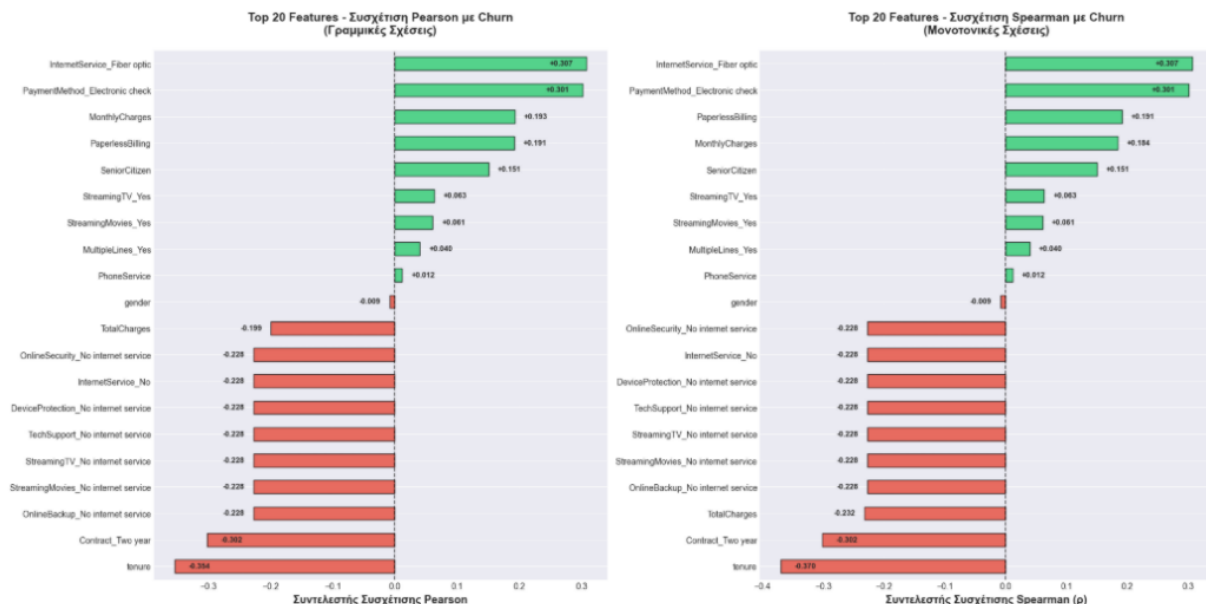


Figure 9: Top 20 features by Pearson (left) and Spearman (right) correlation with churn. The two rankings are nearly identical, a first sign that the relationships are mostly linear.

Feature-Feature correlation analysis: Next, the Pearson correlation matrix between all features was computed to identify strong multicollinearity. A strict threshold (threshold > 0.99)

was applied to detect identical information. Based on this, the following were removed:

- The variable TotalCharges, as it is a derivative of the product MonthlyCharges $\times$ tenure.
- The columns resulting from One-Hot Encoding that concerned the absence of service (e.g. StreamingTV_No_internet_service), as they were perfectly correlated with the central variable InternetService.

In total, 8 columns out of the 30 we had were removed.

```
# Υπολογισμός πίνακα συσχέτισης
corr_matrix = X.corr()
corr_cols = corr_matrix.columns.tolist()
corr_index = corr_matrix.index.tolist()

#Αφαιρούμε όλες τις στήλες με τιμή συσχέτισης μεγαλύτερη του 0.99
cols_to_remove = []
for i in range(len(corr_cols)):
    for j in range(i+1, len(corr_index)):
        corr_value = corr_matrix.iloc[i,j]
        if abs(corr_value) > 0.99:
            if corr_cols[j] not in cols_to_remove:
                cols_to_remove.append(corr_cols[j])
if len(cols_to_remove) > 0:
    print(f"Δεν βρέθηκαν στήλες με συσχέτιση > 0.99")
```

Figure 10: Code: detecting feature pairs with correlation above 0.99 for removal.

Iterative application of VIF (Variance Inflation Factor): For the final multicollinearity check, the VIF method was applied (James et al., 2013). VIF values > 10 indicate a problematic correlation. An iterative procedure was applied:

1. Compute VIF for all features.
2. Identify features with $VIF > 10$.
3. Among the highly correlated features, keep the one with the highest (Pearson) correlation with Churn and remove the rest.
4. Repeat the procedure until all features have $VIF < 10$.

Below is the exact iterative procedure in the code used to apply VIF to the data:

```

# Επαναληπτική διαδικασία αφαίρεσης features με υψηλό VIF
for i in range(MAX_ITERATIONS):
    vif_data = pd.DataFrame()
    vif_data['Feature'] = X_reduced.columns

    # Υπολογισμός VIF για κάθε feature
    vif_data['VIF'] = [variance_inflation_factor(X_reduced.values, j) for j in
range(X_reduced.shape[1])]

    # Έλεγχος αν όλα τα VIF είναι κάτω από το όριο
    if vif_data['VIF'].max() <= 10:
        print("Όλα τα features έχουν VIF <= 10. Δεν απαιτείται περαιτέρω μείωση.")
        break

    # Προσθήκη συσχέτισης με target για απόφαση αφαίρεσης
    pearson_corr = pearson_corr.reset_index().rename(columns={'index': 'Feature',
'Churn': 'Pearson Correlation'})
    vif_data = vif_data.merge(pearson_corr, on='Feature', how='left')
    # Επιλέγουμε όλα τα features με VIF > 10
    features_to_remove = vif_data.loc[vif_data['VIF'] > 10].Feature.tolist()
    # Δημιουργία προσωρινού πίνακα με τα προς αφαίρεση features
    tmp = vif_data.loc[vif_data['Feature'].isin(features_to_remove)]
    # Από αυτά, κρατάμε το feature με τη μεγαλύτερη απόλυτη συσχέτιση με το target
    feature_to_keep = tmp.loc[tmp["Pearson Correlation"].abs().idxmax()]["Feature"]
    # Αφαίρεση του επιλεγμένου feature από τη λίστα αφαίρεσης
    features_to_remove.remove(feature_to_keep)
    # Ενημέρωση λίστας αφαιρεθέντων features
    X_reduced = X_reduced.drop(columns=features_to_remove)
    # Ενημέρωση λίστας αφαιρεθέντων features
    removed_features.extend(features_to_remove)

```

Figure 11: Code: the iterative VIF loop; among correlated features it keeps the one most correlated with churn.

This method ensured that the final dataset is free of multicollinearity, while retaining the strongest predictive features. After the VIF procedure, 2 more columns were removed.

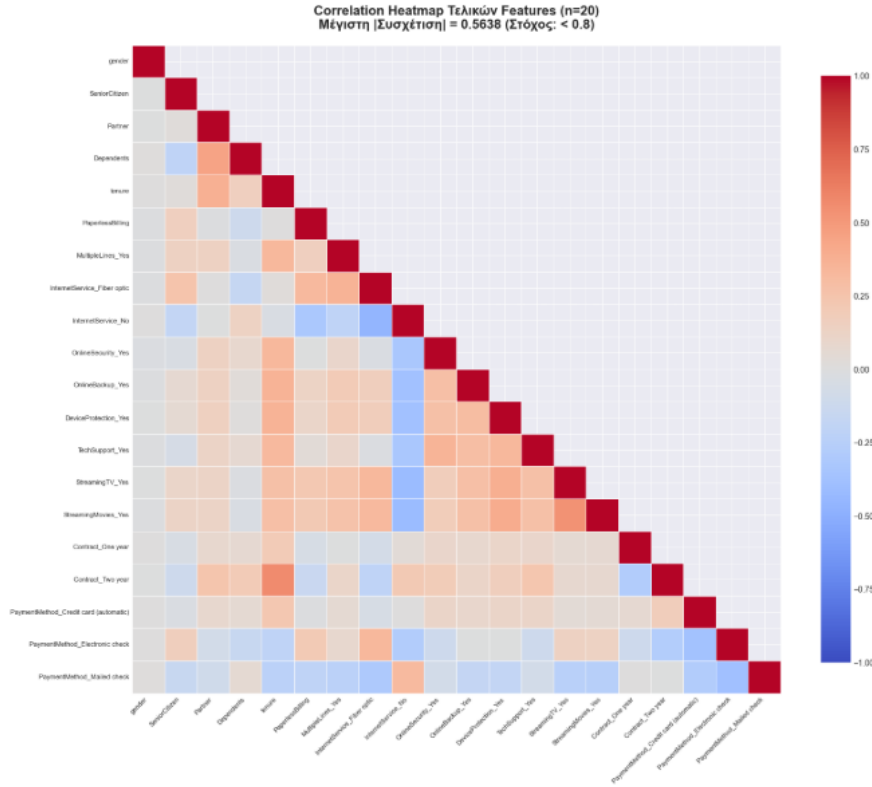


Figure 12: Correlation heatmap of the 20 final features after VIF; the maximum absolute correlation is 0.56, well below the 0.8 target.

1.2.4 Train-test split and data normalisation

The procedure was completed with the splitting and normalisation of the data, in order to prepare them properly for training the models.

Train-Test Split: The data were split into a training set (80%) and a test set (20%). Stratified sampling based on the target variable was used. This ensured that the class ratio (73.4% “No” / 26.6% “Yes”) was kept constant in both subsets, preventing biases during evaluation (He & Garcia, 2009).

```
# Παράμετροι για το train-test split
TEST_SIZE = 0.2
RANDOM_STATE = 42
# Κάνουμε stratified διαχωρισμό
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=TEST_SIZE,
    random_state=RANDOM_STATE,
    stratify=y # -> κρατάει το ποσοστό κάθε κλάσης κατά τον διαχωρισμό.
)
```

Figure 13: Code: stratified 80/20 train-test split preserving the 73.4/26.6 class ratio.

Normalisation: Since all categorical features had already been converted to numeric (via Binary and One-Hot Encoding), normalisation (Goodfellow et al., 2016) was applied using the

StandardScaler on the entire feature set. Specifically:

- The scaler was fitted exclusively on the training data to avoid information leakage.
- Then the transformation was applied to both the training set and the test set.

```
# Δημιουργία του scaler
scaler = StandardScaler()

# Fit & Transform στο training set
X_train_scaled = scaler.fit_transform(X_train)

# Transform μόνο στο test set
X_test_scaled = scaler.transform(X_test)

# Μετατροπή σε πίνακα κρατώντας τα ονόματα στηλών και δείκτες σε train και test sets
X_train_scaled = pd.DataFrame(
    X_train_scaled,
    columns=X_train.columns,
    index=X_train.index
)

X_test_scaled = pd.DataFrame(
    X_test_scaled,
    columns=X_test.columns,
    index=X_test.index
)
```

Figure 14: Code: StandardScaler fitted on the training set only, then applied to both sets.

In this way, all variables acquired a mean value of 0 and a standard deviation of 1. This uniformity is critical for the stability and convergence speed of the optimisation algorithms, as it prevents features with large values from dominating the training process.

2 Development of the Logistic Regression Model

Logistic Regression was chosen as the reference point. The goal was to create a strong comparison baseline, optimised for the particularities of the dataset.

2.1 Optimisation strategy: GridSearchCV

To find the optimal hyperparameters, the GridSearchCV method was used in combination with 5-fold Stratified Cross-Validation (Pedregosa et al., 2011). The search grid included parameters for regularisation (penalty, C), the optimisation algorithm (solver), while the maximum number of iterations (max_iter) was also set, as well as the parameter class_weight='balanced' to handle the class imbalance. The total grid tested 24 hyperparameter combinations.

```

# Ορισμός παραμέτρων για GridSearchCV
param_grid = {
    'penalty': ['l1', 'l2'],                # L1 (lasso) vs L2 (Ridge)
    'C': [0.001, 0.01, 0.1, 1, 10, 100],  # Inverse regularization
    'strength': [0.001, 0.01, 0.1, 1, 10, 100],
    'solver': ['liblinear', 'saga'],        # Solvers που υποστηρίζουν L1
    & L2
    'max_iter': [1000],                    # Μέγιστα iterations (σταθερό)
    'class_weight': ['balanced']           # Αυτόματη προσαρμογή για
}
imbalance

# Ορισμός στρατηγικής cross-validation
cv_strategy = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=RANDOM_STATE
)

# Base estimator
base_estimator = LogisticRegression(random_state=RANDOM_STATE)

# Χρήση F1-Score ως primary optimization metric
f1_scorer = make_scorer(f1_score)

# GridSearchCV setup
grid_search = GridSearchCV(
    estimator=base_estimator,
    param_grid=param_grid,
    scoring=f1_scorer,                      # -> Ισορροπία precision & recall
    cv=cv_strategy,
    n_jobs=-1,                             # Παράλληλη εκτέλεση (όλοι οι πυρήνες)
    verbose=2,                             # Detailed progress
    return_train_score=True,                # Για έλεγχο overfitting
    refit=True                             # Αυτόματο refit με best params
)

# Εκπαίδευση του Grid Search
grid_search.fit(X_train_scaled, y_train)

```

Figure 15: Code: the GridSearchCV setup for logistic regression (24 hyperparameter combinations, 5-fold stratified CV, F1 scoring).

Out of the 24 hyperparameter combinations, the one giving the best mean F1-Score was selected, since it balances Precision and Recall, offering a more reliable picture than plain Accuracy on imbalanced data (He & Garcia, 2009). The hyperparameters that gave the best result were the following:

Hyperparameter	Value
C	0.001
class_weight	Balanced
max_iter	1000
Penalty	L2
solver	saga

Table 1: Best hyperparameters for Logistic Regression.

The best score we obtained on the validation set was $F1 = 0.6303$.

2.2 Results and evaluation

The best model was evaluated on the test set. The following code was used to extract the evaluation metrics:

```

# Κρατάμε το καλύτερο μοντέλο του grid search.
best_model = grid_search.best_estimator_
# Παίρνουμε τα predictions και τις αντίστοιχες πιθανότητες
y_pred = best_model.predict(X_test_scaled)
y_pred_proba = best_model.predict_proba(X_test_scaled)[: , 1]
# Υπολογίζουμε τις μετρικές αξιολόγησης
test_accuracy = accuracy_score(y_test, y_pred)
test_precision = precision_score(y_test, y_pred)
test_recall = recall_score(y_test, y_pred)
test_f1 = f1_score(y_test, y_pred)
test_roc_auc = roc_auc_score(y_test, y_pred_proba)
# και τον Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
tn, fp, fn, tp = cm.ravel()

```

Figure 16: Code: computing the evaluation metrics and confusion matrix on the test set.

The results showed that the model achieved high Recall ($\approx 77\%$), successfully identifying the majority of churning customers, albeit with lower Precision.

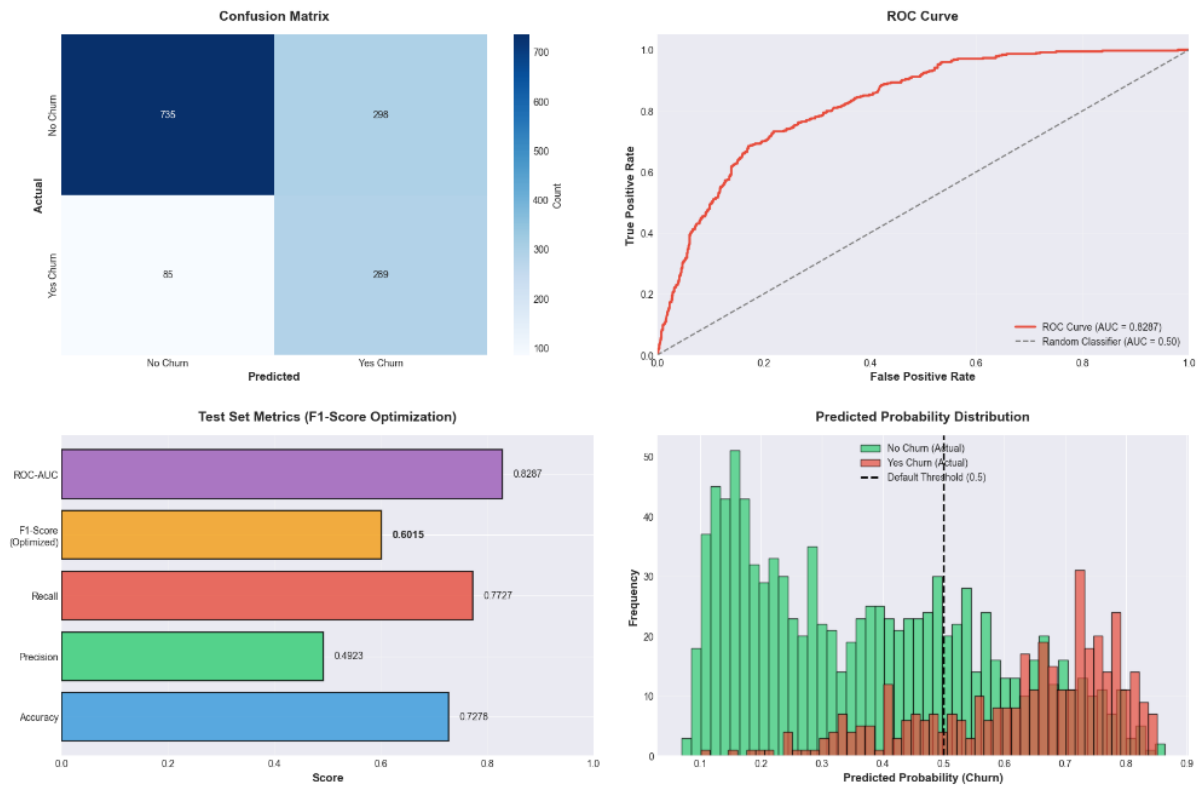


Figure 17: Test-set evaluation of the tuned logistic regression: confusion matrix, ROC curve (AUC 0.8287), the main metrics (F1 0.6015, recall 0.7727), and the predicted probability distribution per class.

Finally, the analysis of the coefficients highlighted the most important factors influencing the churn decision (James et al., 2013), such as monthly contracts and the fiber optic service.

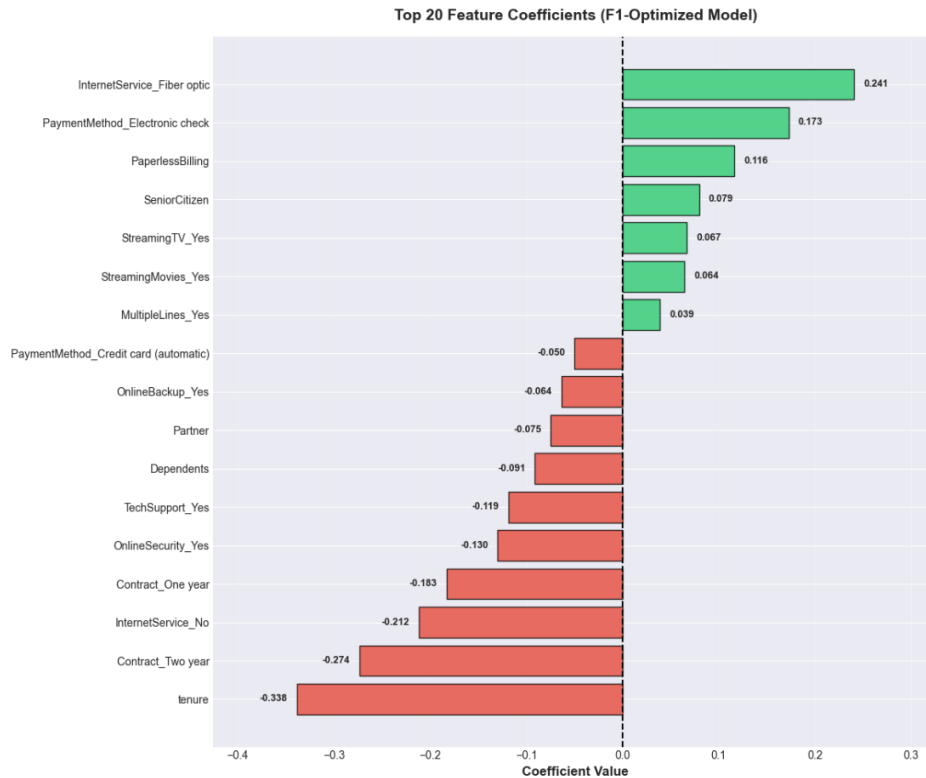


Figure 18: The 20 coefficients of the tuned logistic regression. Fiber optic, electronic check and paperless billing push towards churn; tenure and one/two-year contracts protect against it.

In the following image we can see the final summary for the best logistic regression model.

```
=====
ΤΕΛΙΚΗ ΣΥΝΟΨΗ - LOGISTIC REGRESSION (F1-OPTIMIZED)
=====

Καλύτερη Διαμόρφωση Μοντέλου:
• Μετρική Βελτιστοποίησης: F1-Score
• Penalty: l2
• C (αντίστροφη κανονικοποίηση): 0.001
• Solver: saga
• Μέγιστος αριθμός επαναλήψεων: 1000
• Class weight: balanced

Σύνοψη Απόδοσης:
• CV F1-Score: 0.6303
• Test F1-Score: 0.6015
• Test Precision: 0.4923
• Test Recall: 0.7727
• Test Accuracy: 0.7278
• Test ROC-AUC: 0.8287

Επιχειρησιακή Απόδοση:
• Churners που προβλέφθηκαν: 289/374 (77.3%)
• Churners που χάθηκαν: 85/374 (22.7%)
• False alarms: 298 πελάτες

Στατιστικά Εκπαίδευσης:
• Συνολικοί συνδυασμοί που δοκιμάστηκαν: 24
• Συνολικές προσαρμογές μοντέλου: 120
• Χρόνος εκπαίδευσης: 8.15 δευτερόλεπτα
```

Figure 19: Final summary of the F1-optimised logistic regression: configuration, test performance, business impact (289 of 374 churners caught, 298 false alarms) and training cost (120 fits in 8.15 s).

2.3 Advantages and limitations of the model

The main advantages of logistic regression include computational efficiency and training speed; however, its most important advantage is interpretability. Through the coefficients, it provides direct quantification of how each feature affects the probability of churn (positively or negatively), which is crucial for making business decisions.

On the other hand, the main disadvantage is that the model cannot map complex, non-linear relationships between the features without specialised feature engineering. This limitation makes it necessary to investigate more flexible models, such as Neural Networks, for a possible improvement of predictive ability.

3 Development and Optimisation of the Neural Network

To investigate non-linear relationships in the data, a Multi-Layer Perceptron (MLP) was developed using the TensorFlow/Keras library.

3.1 Architecture and design strategy

The structure of the network was dynamic, allowing the optimisation algorithm to determine its depth and width using Optuna. The general architecture consists of the following parts:

- Input Layer: The network takes as input a feature vector with dimension equal to the number of features after preprocessing.
- Hidden Layers: A variable number of Dense Layers (1 to 3 layers) follows, determined during the optimisation phase. Each hidden layer is structured as follows:
 - Dense Layer: with an optimisable number of neurons.
 - Batch Normalization: for normalisation (Goodfellow et al., 2016).
 - ReLU: for normalisation and faster convergence (Nair & Hinton, 2010).
 - Dropout: for regularisation and avoiding overfitting (Srivastava et al., 2014).
- Output Layer: A dense layer of 2 neurons with a Softmax function to produce the probability distribution of the two classes.

In parallel, L2 Regularization was applied to all dense layers, further enhancing the generalisation of the model (Goodfellow et al., 2016).

```
def build_model(input_dim, hidden_dims, dropout_rate=0.3, l2_reg=1e-4):
    """
    Δημιουργεί και επιστρέφει ένα μοντέλο Keras (Functional API).

    Η αρχιτεκτονική είναι δυναμική και προσαρμόζεται βάσει της λίστας
    'hidden_dims', επιτρέποντας την εύρεση του βέλτιστου βάθους δικτύου.

    Ορίσματα:
        input_dim (int): Το πλήθος των χαρακτηριστικών εισόδου (features).
        hidden_dims (list): Λίστα με τον αριθμό νευρώνων για κάθε κρυφό επίπεδο
        (π.χ. [64, 32]).
        dropout_rate (float): Ποσοστό νευρώνων που απενεργοποιούνται τυχαία (για
        αποφυγή overfitting).
        l2_reg (float): Συντελεστής ποινής L2 (Weight Decay) για τα βάρη.
    """
    # 1. Επίπεδο Εισόδου (Input Layer)
    # Καθορίζουμε το σχήμα των δεδομένων που θα δεχτεί το δίκτυο (διάνυσμα μεγέθους
    input_dim).
    inputs = layers.Input(shape=(input_dim,), name='input')
    x = inputs

    # 2. Δημιουργία Κρυφών Επιπέδων (Hidden Layers)
    # Χρησιμοποιούμε βρόχο για να προσθέσουμε δυναμικά όσα επίπεδα ορίζει το Optuna.
    for i, hidden_dim in enumerate(hidden_dims):
        # α) Πυκνό Επίπεδο (Dense) με L2 Regularization
        # Το L2 προσθέτει ποινή στα μεγάλα βάρη, αποτρέποντας την υπερβολική
        εξάρτηση από συγκεκριμένα χαρακτηριστικά.
        x = layers.Dense(
            hidden_dim,
            kernel_regularizer=regularizers.l2(l2_reg),
            name=f'dense_{i+1}'
        )(x)
```

Figure 20: Code: the dynamic Keras model builder; input layer and hidden dense layers with L2 regularization.

```
        # β) Batch Normalization
        # Κανονικοποιεί τις εξόδους του προηγούμενου επιπέδου πριν την ενεργοποίηση.
        # Αυτό σταθεροποιεί την εκπαίδευση και επιτρέπει υψηλότερο learning rate.
        x = layers.BatchNormalization(name=f'bn_{i+1}')(x)

        # γ) Συνάρτηση Ενεργοποίησης ReLU
        # Εισάγει τη μη-γραμμικότητα στο μοντέλο, επιτρέποντας την εκμάθηση σύνθετων
        μοτίβων.
        # Εφαρμόζεται META το Batch Norm που είναι βέλτιστη πρακτική.
        x = layers.Activation('relu', name=f'relu_{i+1}')(x)

        # δ) Dropout
        # Απενεργοποιεί τυχαία ένα ποσοστό νευρώνων σε κάθε βήμα εκπαίδευσης.
        # Είναι η κύρια γραμμή άμυνας ενάντια στο overfitting.
        x = layers.Dropout(dropout_rate, name=f'dropout_{i+1}')(x)

    # 3. Επίπεδο Εξόδου (Output Layer)
    # Χρησιμοποιούμε 2 νευρώνες με Softmax για να πάρουμε πιθανότητες για κάθε κλάση
    (Churn / No Churn).
    # Το άθροισμα των δύο πιθανοτήτων θα είναι πάντα 1.
    outputs = layers.Dense(2, activation='softmax', name='output')(x)

    # 4. Τελική Σύνθεση Μοντέλου
    # Συνδέουμε την είσοδο με την έξοδο σε ένα αντικείμενο Model.
    model = Model(inputs=inputs, outputs=outputs, name='churn_nn')
    return model
```

Figure 21: Code: batch normalisation, ReLU and dropout inside each hidden layer, and the two-neuron softmax output.

3.2 Hyperparameter optimisation with Optuna

Optuna utilises the Tree-structured Parzen Estimator (TPE) algorithm to explore the hyperparameter space more efficiently (Akiba et al., 2019).

3.2.1 Definition of the search space

The search space was designed to cover a wide range of architectural choices:

- Network structure: 1–3 hidden layers and 32 to 256 neurons per layer.
- Learning rate: Range from 0.0001 to 0.01 on a logarithmic scale.
- Regularisation: Dropout from 0.1 to 0.5 and L2 Regularization strength.
- Batch Size: 16, 32, 64, 128.

The models were trained using the Adam optimiser (Kingma & Ba, 2014), because of its efficiency in deep learning problems. The training procedure was further enhanced with the use of early stopping to limit overfitting (Prechelt, 1998) and the ReduceLROnPlateau mechanism for adjusting the learning rate, ensuring the optimal minimisation of the error (Goodfellow et al., 2016).

```
# Ορισμός διαστημάτων για τις υπερπαραμέτρους (Search Space)
n_layers = trial.suggest_int('n_layers', 1, 3)
# Δυναμικός ορισμός νευρώνων για κάθε επίπεδο
hidden_dims = []
for i in range(n_layers):
    hidden_dims.append(trial.suggest_int(f'n_units_{i}', 32, 256, log=True))
# Υπόλοιπες υπερπαραμέτροι
dropout_rate = trial.suggest_float('dropout_rate', 0.1, 0.5)
learning_rate = trial.suggest_float('learning_rate', 1e-4, 1e-2, log=True)
batch_size = trial.suggest_categorical('batch_size', [16, 32, 64, 128])
l2_reg = trial.suggest_float('l2_reg', 1e-5, 1e-3, log=True)
# Οριστικοποίηση του μοντέλου με Βελτιστοποιητή Adam
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy', F1Score()]
)
# Εκπαίδευση του μοντέλου, με πρόωρη διακοπή και ReduceLROnPlateau, αποθηκεύοντας
την ιστορία εκπαίδευσης στην μεταβλητή history
history = model.fit(
    X_fold_train, y_fold_train,
    validation_data=(X_fold_val, y_fold_val),
    epochs=100,
    batch_size=batch_size,
    class_weight=class_weights,
    callbacks=[early_stop, reduce_lr],
    verbose=0
)
```

Figure 22: Code: the Optuna search space and the training call with class weights, early stopping and ReduceLROnPlateau.

3.2.2 Evaluation strategy and pruning

To ensure the robustness of the results, each Optuna trial was evaluated via 5-fold Stratified Cross-Validation. The objective function aimed at maximising the mean F1-Score across the 5

folds. In addition, the Median Pruner mechanism was activated.

```
# Ορισμός μελετητή Optuna
study = optuna.create_study(
    direction='maximize',
    study_name='churn_nn_f1_optimization',
    pruner=MedianPruner(n_startup_trials=10, n_warmup_steps=2), # -> Για γρήγορο
    sampler=optuna.samplers.TPESampler(seed=RANDOM_STATE)
)
```

Figure 23: Code: the Optuna study with the TPE sampler and the Median Pruner for early termination of weak trials.

This technique terminates early the trials that, in the first stages of training, perform worse than the median of the previous trials, saving significant computational time.

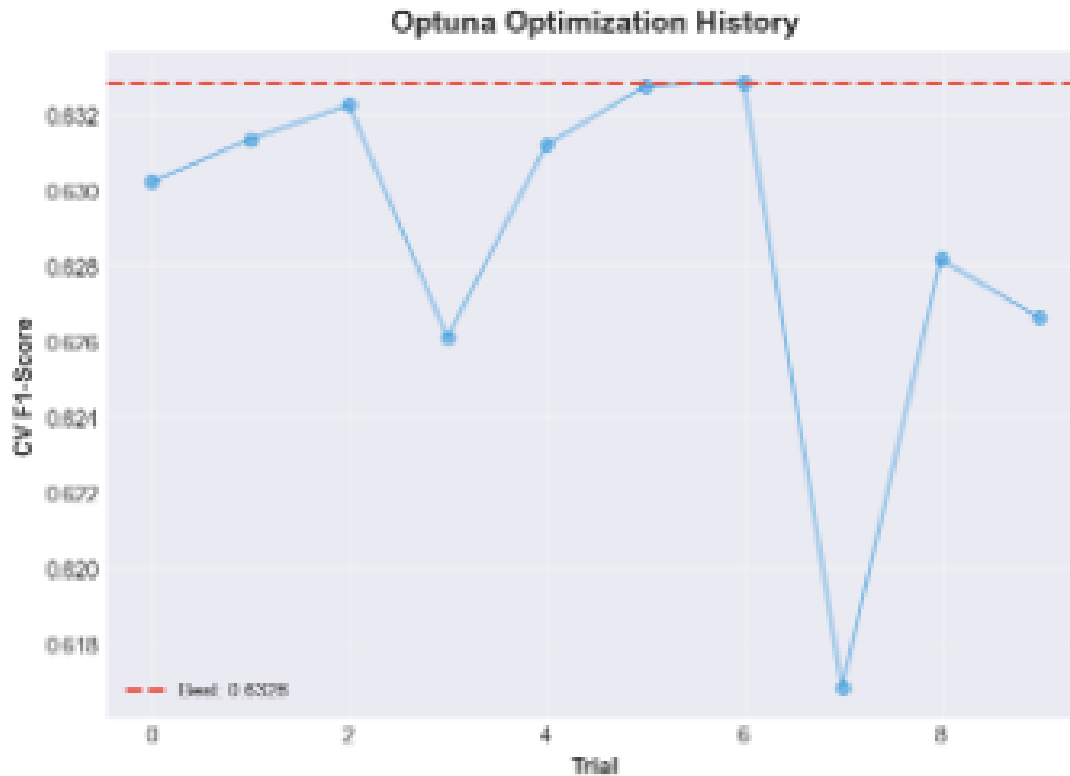


Figure 24: Optuna optimisation history: cross-validated F1 per trial, with the best value (0.6328) marked by the dashed line.

3.2.3 Optimisation results

The procedure showed that a simpler architecture was more efficient than deeper networks, avoiding overfitting. The final hyperparameter selection is shown below:

Hyperparameter	Value
Hidden layers	1
Neurons	62
Dropout	0.255
Learning rate	0.0003
Batch size	16
L2_reg	0.00002

Table 2: Best hyperparameters for the Neural Network.

3.3 Handling class imbalance

Given the significant imbalance in the dataset (73% No, 27% Yes), two different strategies were investigated to enhance the network's ability to detect churning customers.

- Class Weights: Greater weight was applied to the prediction errors of the minority class (Churn=Yes) when computing the loss function.
- SMOTE technique: Synthetic training data were generated for the minority class, balancing the sample to a 1:1 ratio (Chawla et al., 2002).

```
# Αντιμετώπιση ανισορροπίας των κλάσεων

# Τρόπος 1. Εκπαίδευση μοντέλου με class weights
if class_weights:
    print("Using class weights")
    history = model.fit(
```

Figure 25: Code: strategy 1 for class imbalance, training with class weights.

```

X_fold_train, y_fold_train,
validation_data=(X_fold_val, y_fold_val),
epochs=100,
batch_size=batch_size,
class_weight=class_weights,
callbacks=[early_stop, reduce_lr],
verbose=0
)
# Τρόπος 2. Ορισμός SMOTE για την εξισορρόπηση των κλάσεων
smote = SMOTE(
    sampling_strategy='auto', # Balance to 1:1
    k_neighbors=5, # Default: 5 nearest neighbors
    random_state=RANDOM_STATE
)
# Εφαρμογή SMOTE
X_train_smote, y_train_smote = smote.fit_resample(X_train_scaled, y_train)
# Μετατροπή πίσω σε DataFrame/Series για συνέπεια
X_train_smote = pd.DataFrame(
    X_train_smote,
    columns=X_train_scaled.columns
)
y_train_smote = pd.Series(y_train_smote, name='Churn')
# Στη συνέχεια κάνουμε το train πάνω στα smote δεδομένα χωρίς να χρησιμοποιούμε
class weights αφού πλέον έχουμε εξισορροπήσει τις κλάσεις.

```

Figure 26: Code: strategy 2 for class imbalance, SMOTE resampling of the training set to a 1:1 ratio.

From the comparative evaluation it emerged that the use of SMOTE brought no improvement over the Class Weights method. This is probably due to the fact that SMOTE, by creating artificial samples through interpolation, introduced noise at the decision boundaries of the two classes. Consequently, for the final model the Class Weights approach was adopted, which preserved the integrity of the original data.

3.4 Evaluation of the final Neural Network

The best model resulting from the Optuna procedure was retrained on the full training set and evaluated on the independent test set. The results are shown below in comparison with the baseline model:

- F1-Score: 61.89% (versus 60.15% for Logistic Regression).
- Recall: 80.75% (an increase of +3.5% over Logistic Regression).
- ROC-AUC: 83.51%.

The improvement of about 1.74% in F1-Score and the small increase in Recall indicate that there were no serious non-linear correlations between the data, since if there were, the differences would be considerably higher.

The study of the learning curves shows smooth convergence, with the performance on the validation set closely following that of the training set, confirming the successful prevention of overfitting thanks to the use of Dropout and L2 Regularization.

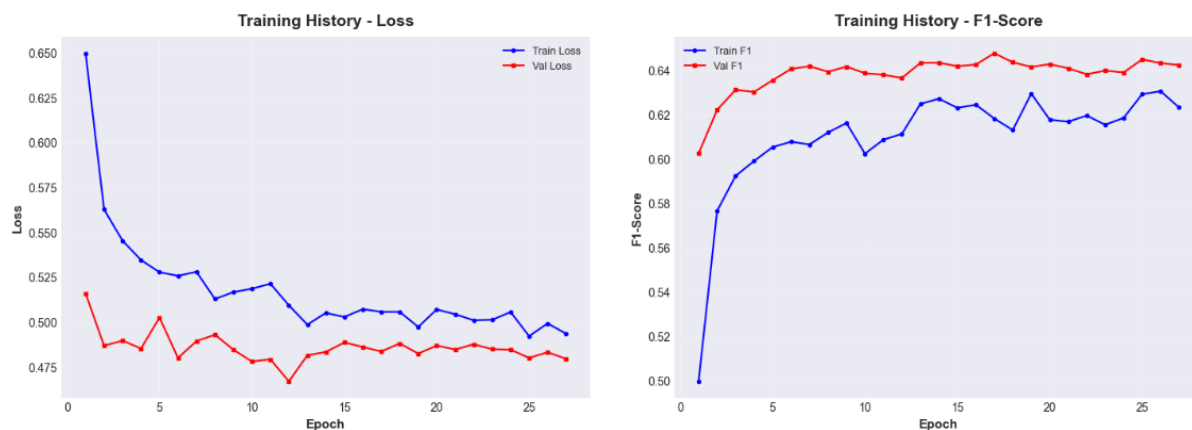


Figure 27: Learning curves of the final neural network: training and validation loss and F1 converge smoothly, with no overfitting gap.

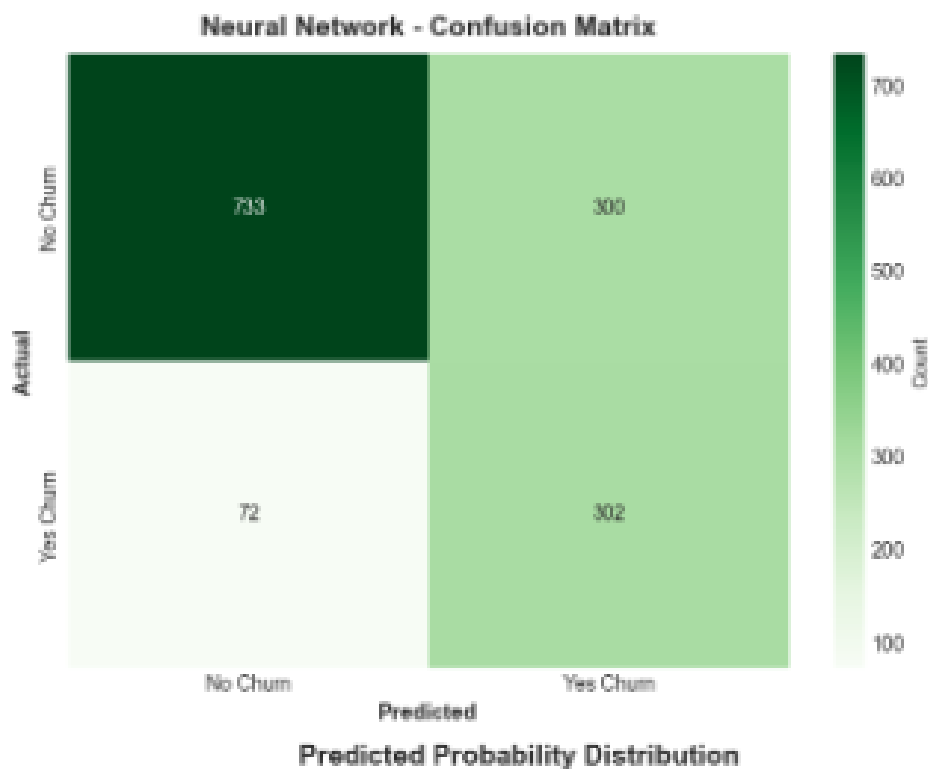


Figure 28: Confusion matrix of the neural network on the test set: 302 of 374 churners detected (recall 80.75%).

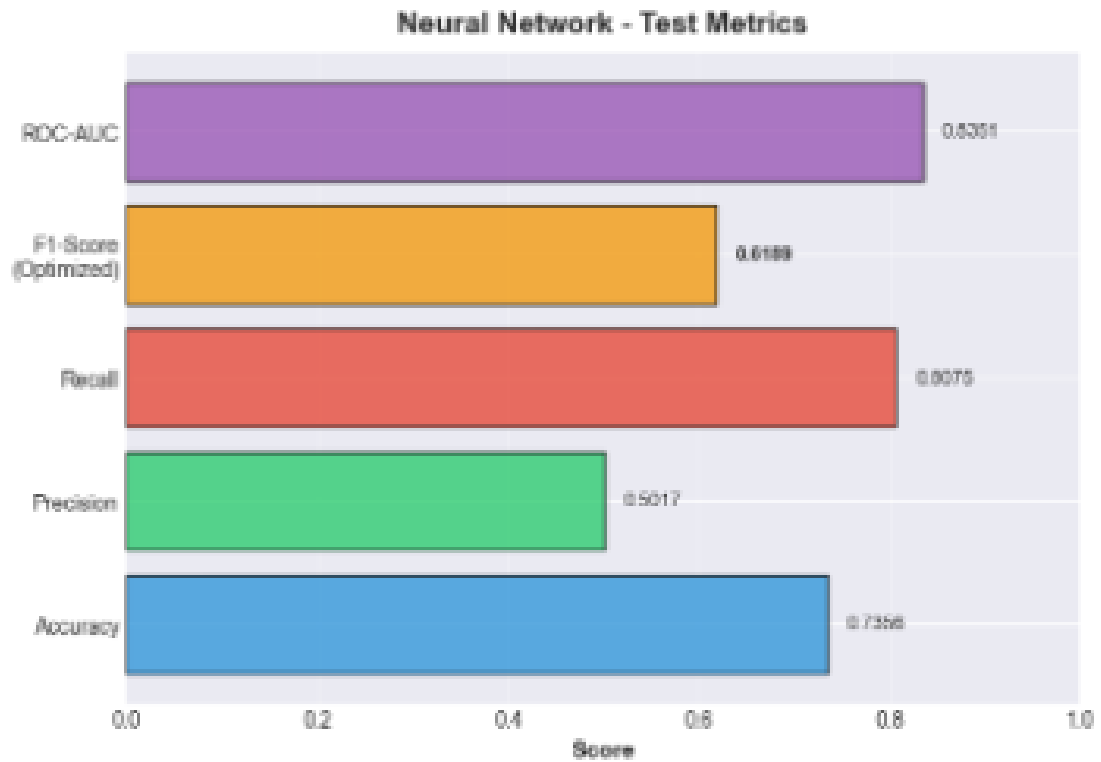


Figure 29: Test metrics of the neural network: ROC-AUC 0.8351, F1 0.6189, recall 0.8075, precision 0.5017, accuracy 0.7356.

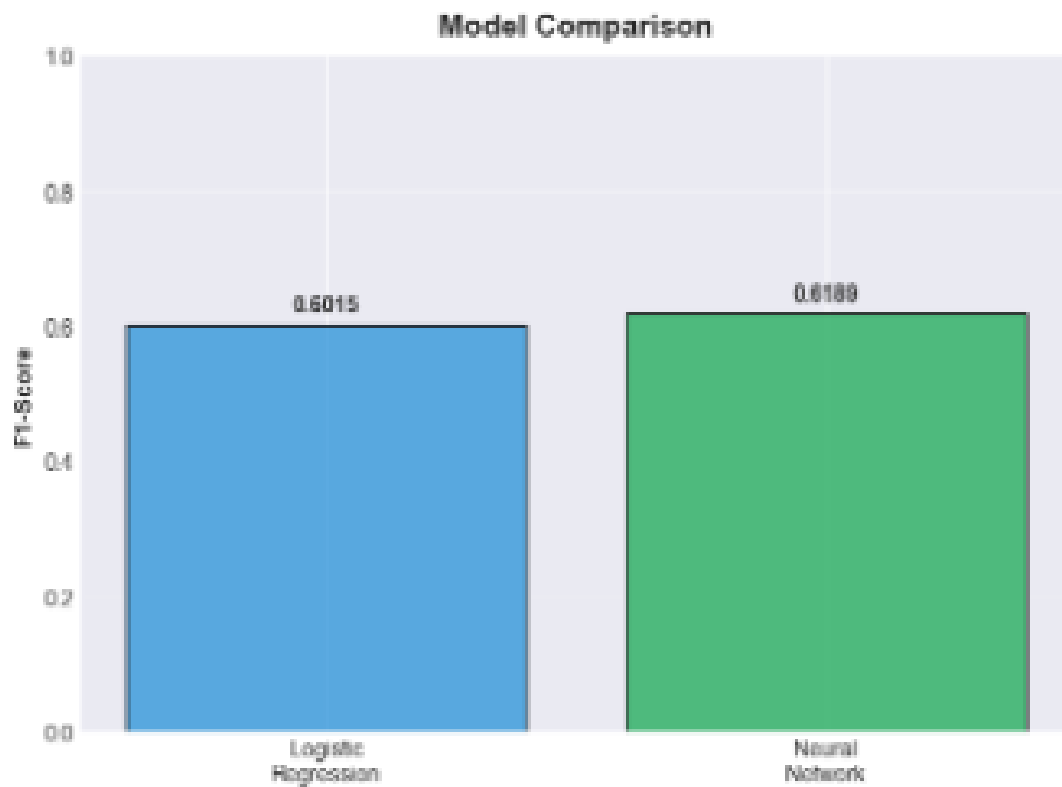


Figure 30: F1 comparison of the tuned logistic regression (0.6015) and the neural network (0.6189).

4 Advanced Modelling with CatBoost

In order to exhaust the margins for improving predictive ability, the Categorical Boosting (CatBoost) algorithm was applied (Prokhorenkova et al., 2018). CatBoost was chosen because it excels at handling categorical data without the need for extensive preprocessing and is extremely robust to overfitting.

4.1 Optimisation and training

The same strict optimisation methodology was followed with Optuna (5-fold Stratified CV), aiming at maximising the F1-Score. The tuned hyperparameters included the depth of the trees (depth), the learning rate (learning_rate), the number of iterations (iterations) and the L2 regularisation on the leaves (l2_leaf_reg). In addition, CatBoost’s ability to handle categorical variables automatically (cat_features) was exploited, allowing the model to identify complex non-linear interactions that are missed by classical encoding methods.

```
# Δημιουργία Catboost model για χρήση στο optuna.
def build_model(params):
    return CatBoostClassifier(
        iterations=params['iterations'],
        learning_rate=params['learning_rate'],
        depth=params['depth'],
        l2_leaf_reg=params['l2_leaf_reg'],
        random_seed=RANDOM_STATE,
        eval_metric='F1',
        loss_function='Logloss',
        task_type='GPU',
        verbose=0,
        early_stopping_rounds=20,
        class_weights=class_weights_dict
    )
```

Figure 31: Code: the CatBoost model builder used inside the Optuna objective (GPU training, F1 eval metric, early stopping).

```

# Ορισμός υπερπαραμετρικού χώρου -> μέρος του objective function
params = {
    'iterations': trial.suggest_int('iterations', 100, 1000, step=100),
    'learning_rate': trial.suggest_float('learning_rate', 1e-3, 0.3, log=True),
    'depth': trial.suggest_int('depth', 4, 10),
    'l2_leaf_reg': trial.suggest_float('l2_leaf_reg', 1, 10),
}

# Cross-validation με StratifiedKfold -> ώστε κάθε φορά να έχουμε σταθερή αναλογία
κλάσεων
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

# 4. Δημιουργία Αντικειμένου Μελέτης (Study)
study_catboost = optuna.create_study(
    direction='maximize', # Στόχος: Μεγιστοποίηση του F1-Score
    study_name='catboost_churn_f1_optimization',
    # Μαθαίνει από προηγούμενα trials για να προτείνει καλύτερες παραμέτρους
    sampler=optuna.samplers.TPESampler(seed=RANDOM_STATE),
    # Median Pruner: Μηχανισμός πρόωρης διακοπής
    pruner=MedianPruner(n_startup_trials=5, n_warmup_steps=10)

# Έναρξη βελτιστοποίησης με το optuna
study_catboost.optimize(objective, n_trials=N_TRIALS, timeout=3600,
show_progress_bar=True)

```

Figure 32: Code: the CatBoost hyperparameter search space and the Optuna study with 5-fold stratified CV.

4.2 Results and comparison

The optimised CatBoost model recorded the top performance among all the algorithms tested:

- F1-Score: 62.47% (higher than the 61.89% of the Neural Network and the 60.15% of LogReg).
- ROC-AUC: 83.60% (the highest discriminative ability).
- Accuracy: 75.41%.

Although the difference in F1-Score compared to the Neural Network is small (+0.6%), CatBoost offers significant advantages: faster training, stability and, above all, the ability of direct interpretation.

The model's evaluation diagrams are shown below.

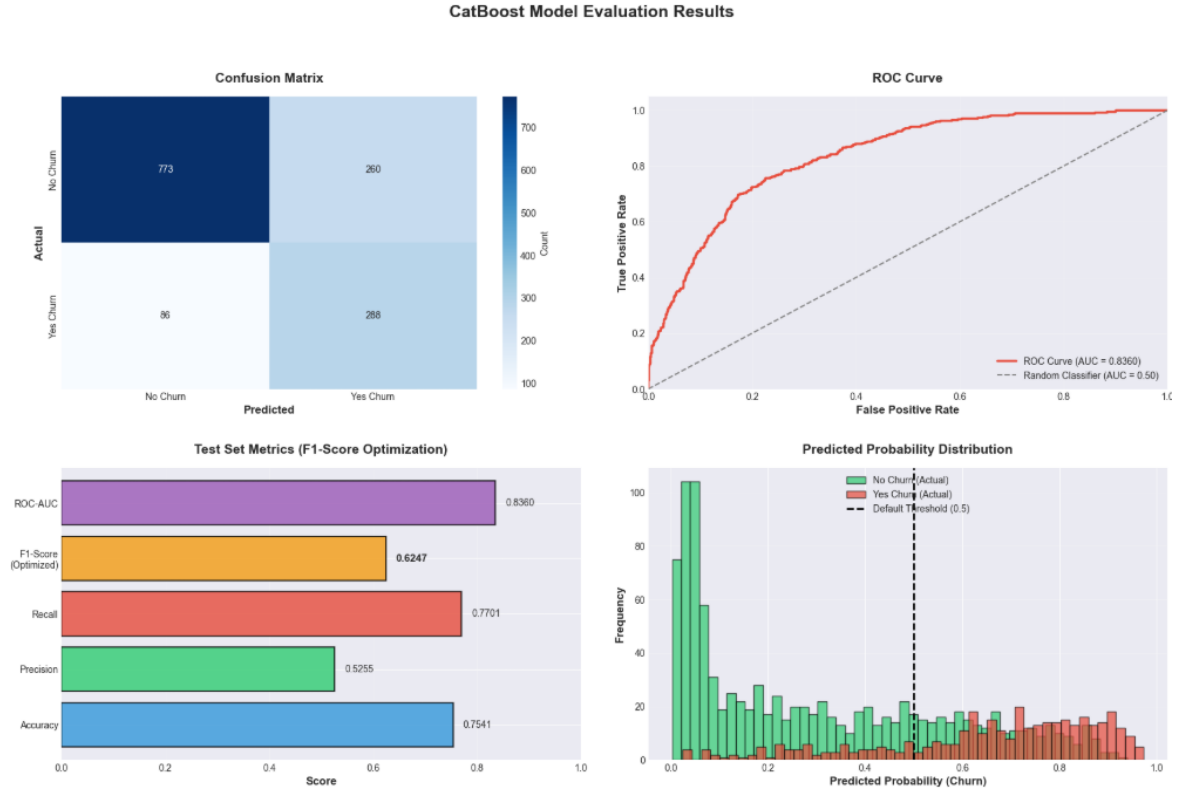


Figure 33: Test-set evaluation of the tuned CatBoost model: confusion matrix, ROC curve (AUC 0.8360), the main metrics (F1 0.6247, recall 0.7701), and the predicted probability distribution per class.

5 Comparative Analysis and Final Model Selection

At this stage, the overall evaluation of the four experimental setups that were developed is carried out:

- Logistic Regression: Linear model with F1 optimisation.
- Neural Network (Class Weights): MLP with imbalance handling via weights.
- Neural Network (SMOTE): MLP with imbalance handling via synthetic data.
- CatBoost: Gradient Boosting with native support for categorical data and F1 optimisation.

5.1 Performance comparison

5.1.1 Predictive performance

The comparison was based primarily on the F1-Score and secondarily on Recall and ROC-AUC.

F1-Score: The CatBoost model recorded the highest performance (62.47%), closely followed by the Neural Network with Class Weights (61.89%). Logistic Regression lagged slightly behind (60.15%).

Impact of SMOTE: Applying SMOTE to the Neural Network led to a decrease in performance (F1: ~58–59%) compared to using Class Weights. This indicates that the artificial

inflation of the sample introduced noise and overlap between the classes, making discrimination harder for the model.

Recall vs Precision: The Neural Network (Class Weights) achieved the highest Recall (~80%), detecting the most customers who leave; however, this came at the expense of Precision, leading to more false positives. CatBoost offered the most balanced behaviour.

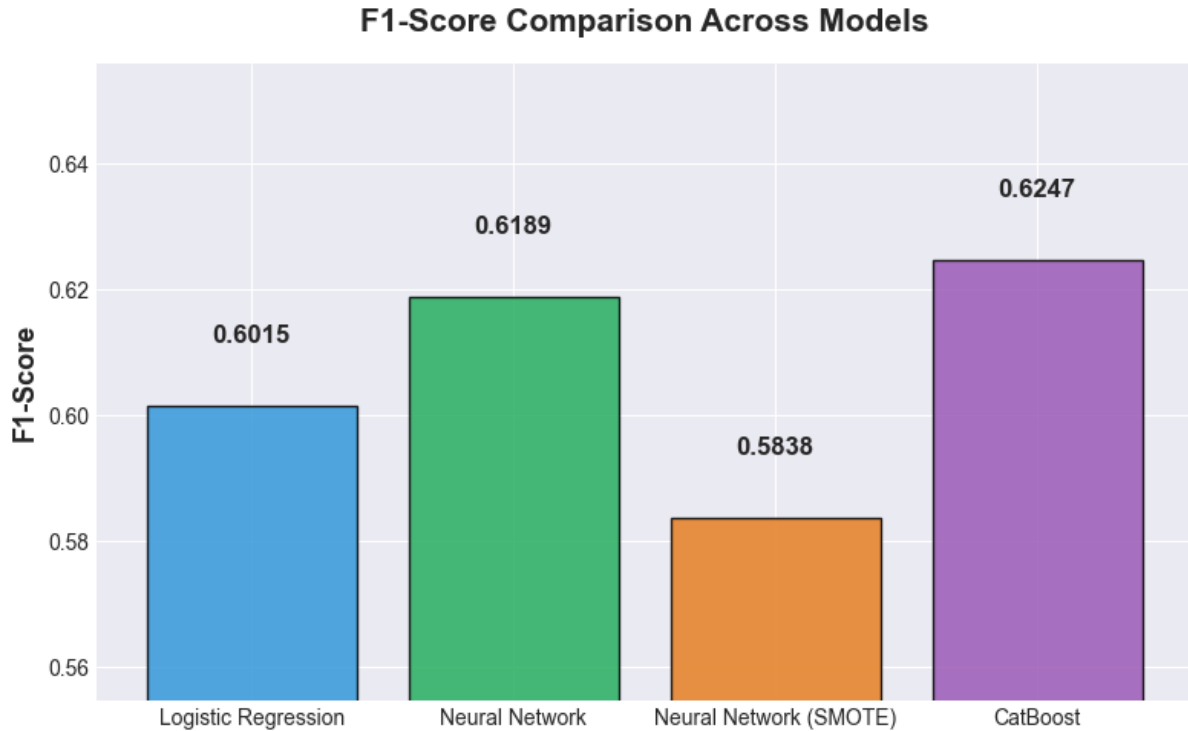


Figure 34: F1-score comparison across the four setups: CatBoost 0.6247, neural network 0.6189, logistic regression 0.6015, neural network with SMOTE 0.5838.

Below is the final summary of the evaluation on the most basic classification metrics:

	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.7278	0.4923	0.7727	0.6015	0.8287
Neural Network	0.7356	0.5017	0.8075	0.6189	0.8351
Neural Network (SMOTE)	0.7406	0.5089	0.6845	0.5838	0.7997
CatBoost	0.7541	0.5255	0.7701	0.6247	0.8360

Figure 35: Full metric comparison of the four setups on the test set (accuracy, precision, recall, F1, ROC-AUC).

5.1.2 Computational cost and training times

A critical factor for business application is the required time and resources.

- Logistic Regression: Proved extremely efficient, completing 120 fits in GridSearch in just 8.15 seconds. It requires minimal memory and runs comfortably on a plain CPU.

- CatBoost: The optimisation procedure (10 trials with Optuna) lasted 3.8 minutes. This represents an increase in development time of approximately 335 times compared to the baseline model.
- Neural Networks: Required the most time and benefit significantly from the use of a GPU, making them the most expensive in computational resources.

5.1.3 Final selection and critical evaluation

The small deviation in performance ($\sim 2\%$ in F1-Score) between simple Logistic Regression and the complex non-linear models (Neural Networks, CatBoost) is a strong indication that the relationships governing customer churn are mainly linear; otherwise Logistic Regression would show a significant lag, which was not observed.

Therefore, for the present phase and this specific dataset, Logistic Regression emerges as the most rational choice. It offers:

- More than adequate predictive ability (only marginally lower than the best).
- Maximum speed and minimal computational cost.
- Full transparency and interpretability.

However, the linear model is not flexible enough. In a future phase of the project, where we would request additional customer information from the company, the data would be enriched (e.g. with unstructured data, complaint logs), introducing complexity that may exceed the capabilities of a linear model. CatBoost is therefore chosen as the most flexible solution for incorporating future information.

6 Model Interpretability with SHAP

To address the model's lack of transparency and extract actionable conclusions, the SHAP method (SHapley Additive exPlanations) was applied, which is based on game theory to quantify the contribution of each feature to the final prediction (Lundberg & Lee, 2017).

6.1 Global feature importance

The TreeExplainer was used, which is optimised for tree models, to compute the SHAP values that quantify the contribution of each feature.

```

# Δημιουργία explainer
explainer = shap.TreeExplainer(cat_model)
# Υπολογισμός SHAP values πάνω στο training set
shap_values_full = explainer.shap_values(X_train_cb)

# Αν το shap_values_full είναι λίστα, τότε επιλέγουμε τα values του positive case
# (Churn=True)
if isinstance(shap_values_full, list):
    shap_values = shap_values_full[1]
# Αλλιώς επιλέγουμε όλα τα values
else:
    shap_values = shap_values_full

# Δημιουργία Beeswarm plot
shap.summary_plot(
    shap_values,
    X_train_cb,
    feature_names=X_train_cb.columns,
    show=False
)

# Δημιουργία ραβδογράμματος μέσης απόλυτης επίδρασης ανα χαρακτηριστικό.
shap.summary_plot(
    shap_values,
    X_train_cb,
    feature_names=X_train_cb.columns,
    plot_type="bar",
    show=False
)

```

Figure 36: Code: computing SHAP values with the TreeExplainer and producing the beeswarm and bar summary plots.

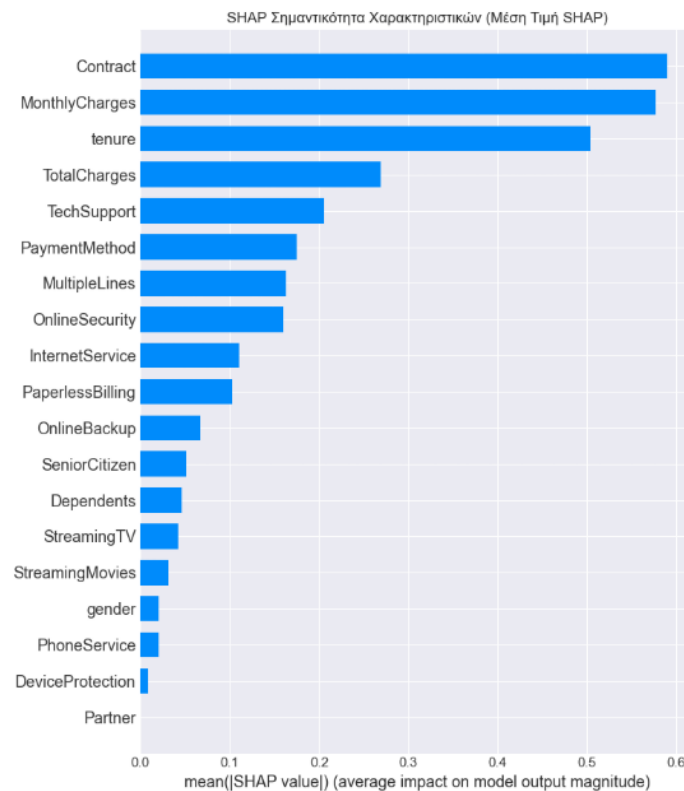


Figure 37: Global SHAP feature importance (mean absolute SHAP value): Contract, Monthly-Charges and tenure dominate the churn decision.

As it turns out, the three dominant factors determining the churn decision are the type of contract (Contract), the amount of monthly charges (MonthlyCharges) and the duration of stay (tenure).

6.2 Direction and nature of the effect

For a deeper understanding of the relationship of the variables with Churn, the summary plot was examined. Here, each dot represents a customer, while the colour indicates the value of the feature (Red = High, Blue = Low, Grey for categorical).

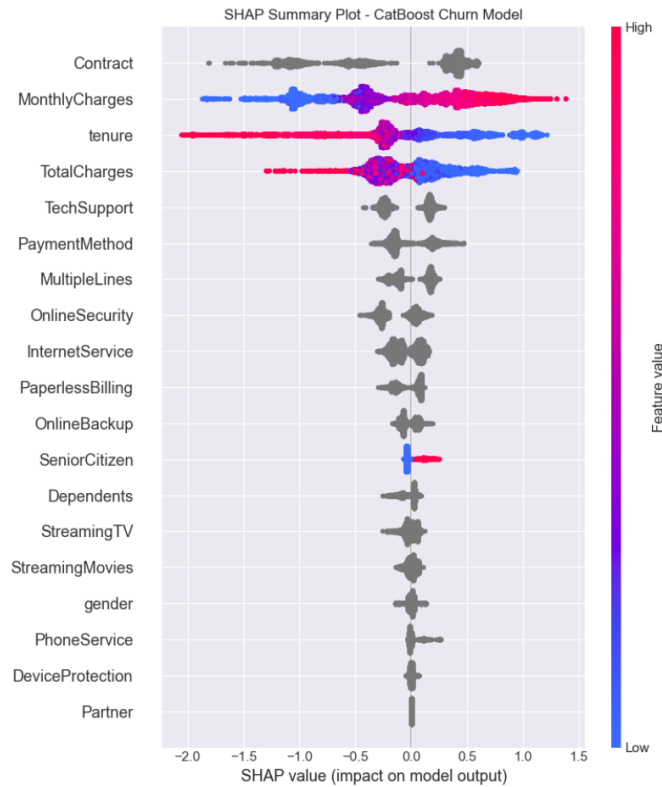


Figure 38: SHAP summary (beeswarm) plot for the CatBoost model. Each dot is a customer; red means a high feature value, blue a low one, grey a categorical feature.

The findings are revealing:

- **Contract:** Customers with short-term contracts (Month-to-month) have strongly positive SHAP values (right side), dramatically increasing the probability of leaving.
- **Tenure:** A strong negative correlation is observed. “Old” customers (red dots) are concentrated on the left, which means that a long stay reduces the risk of churn.
- **Internet Service:** Fiber Optic users show an increased tendency to leave, possibly due to cost or technical issues.

6.3 Local interpretation

To demonstrate the model’s ability to explain individual predictions, a random customer sample was analysed via the Waterfall Plot.

```

# Επιλέγουμε ένα συγκεκριμένο sample (π.χ. index 0)
sample_idx = 0
x_sample = X_train_cb.iloc[[sample_idx]] # Διατήρηση 2D shape

# Υπολογίζουμε ξανά SHAP values μόνο για αυτό το sample
shap_sample = explainer.shap_values(x_sample)

# Αν είναι list, παίρνουμε την positive class
if isinstance(shap_sample, list):
    shap_sample = shap_sample[1]

# Αναμενόμενη τιμή για την positive class
expected_values = explainer.expected_value

# Αν έχουμε binary case κρατάμε μόνο το positive class
if isinstance(expected_values, (list, np.ndarray)):
    base_val = expected_values[1] # positive class
# Αλλιώς κρατάμε την αναμενόμενη τιμή όπως είναι
else:
    base_val = expected_values

# Δημιουργία Waterfall plot
shap.plots._waterfall.waterfall_legacy(
    expected_values,
    shap_sample.flatten(),
    x_sample.iloc[0, :].values,
    feature_names=X_train_cb.columns
)

```

Figure 39: Code: computing SHAP values for a single customer and building the waterfall plot.

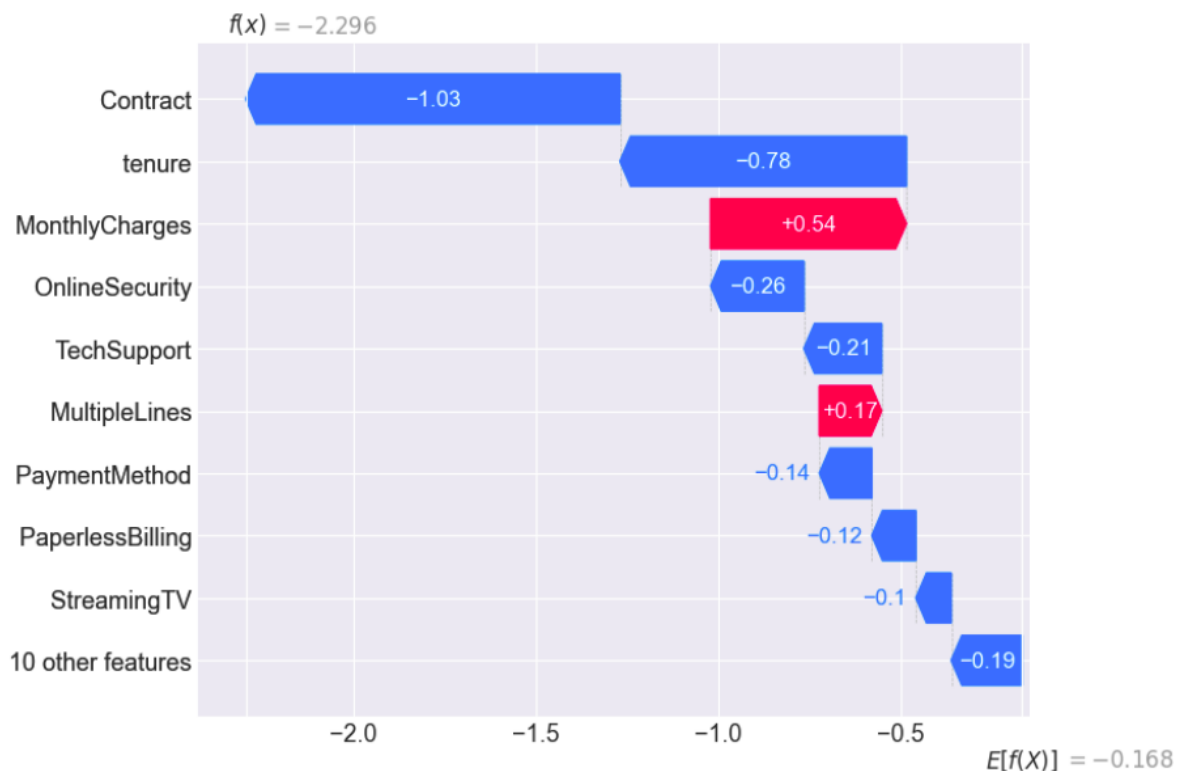


Figure 40: Waterfall plot for one customer: starting from the base value $E[f(X)] = -0.168$, each feature pushes the prediction up (red) or down (blue) to $f(x) = -2.296$.

The diagram starts from the average churn probability and shows how each feature increases

(red) or decreases (blue) the risk for the specific customer. This visualisation helps the customer service department to tailor targeted actions, such as offering a discount when MonthlyCharges is a major risk factor.

7 Conclusions and Future Directions

The present study completed the development cycle of a customer churn prediction system, highlighting the CatBoost model as the optimal choice. The detailed examination of the Confusion Matrix reveals the strengths and weaknesses of the model:

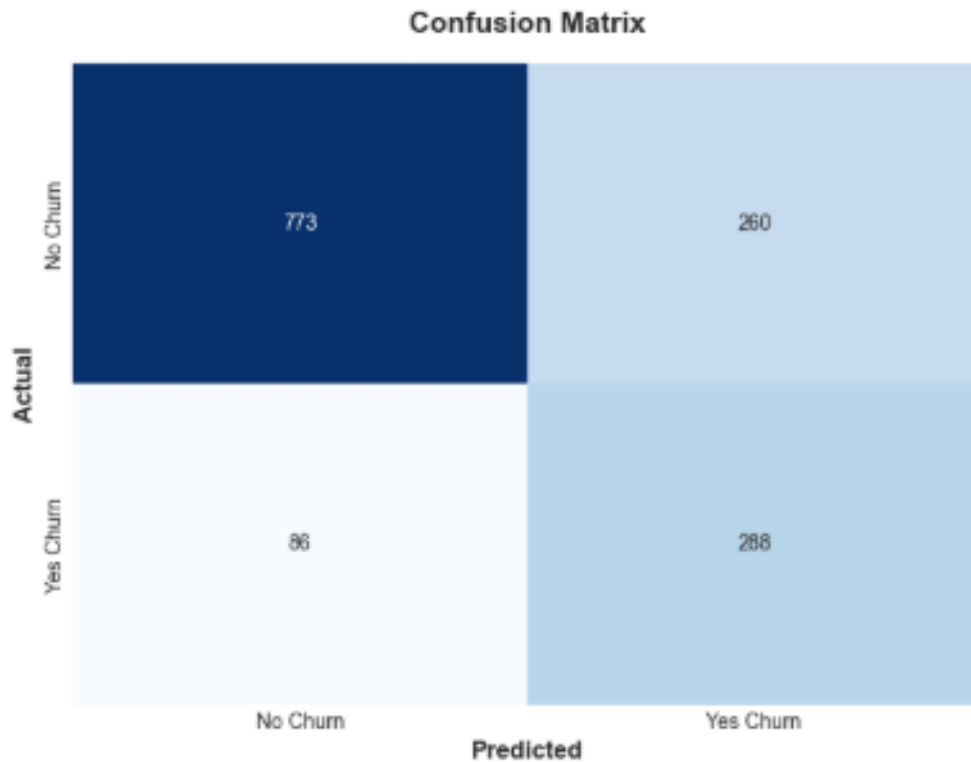


Figure 41: Confusion matrix of the final CatBoost model: 288 of 374 churners caught, 260 false alarms.

Through the SHAP analysis we were able to identify the main factors that drive customers away. These were flexible contracts, high monthly charges and service quality, with the fiber optic connection often appearing as a source of dissatisfaction.

Based on the above, the following actions are proposed for the company:

- **Contract Migration Strategy:** Targeting customers with monthly contracts and offering discounts for switching to an annual commitment.
- **Fiber Optic Quality Control:** Immediate investigation of complaints related to fiber optics and possible readjustment of the pricing policy.
- **Managing False Positives:** Due to the many False Positives, retention campaigns should be low-cost (e.g. newsletters, preferential service) and not aggressive discounts that would unnecessarily reduce the profit margin on loyal customers.

To optimise the model and reduce unnecessary costs, certain future steps are required. First, the performance of the model depends on the quality of the data, so enriching it with new features, such as call history, usage logs and complaint texts, would allow the detection of behavioural patterns beyond basic demographic data. Second, increasing the number of customers in the dataset would strengthen the performance of boosting models, which scale particularly well to tens or hundreds of thousands of records. Finally, exploring more advanced approaches, such as combining neural networks with gradient boosting, could further reduce classification errors.

8 Bibliography

1. Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (pp. 2623–2631).
2. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
3. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT press.
4. Hancock, J. T., & Khoshgoftaar, T. M. (2020). Survey on categorical data for neural networks. *Journal of Big Data*, 7(1), 1–41.
5. He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284.
6. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An introduction to statistical learning*. Springer.
7. Kaggle. (n.d.). Telco Customer Churn. Retrieved November 2025, from <https://www.kaggle.com/blastchar/telco-customer-churn>
8. Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
9. Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.
10. Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning*.
11. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
12. Prechelt, L. (1998). Early stopping-but when? In *Neural Networks: Tricks of the Trade* (pp. 55–69). Springer, Berlin, Heidelberg.

13. Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. *Advances in Neural Information Processing Systems*, 31.
14. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1), 1929–1958.

A Explanations of Methods and Algorithms

A.1 Explanation of the Variance Inflation Factor

The Variance Inflation Factor (VIF) is a statistical tool used to detect multicollinearity in a dataset. Multicollinearity occurs when two or more independent variables are strongly correlated with each other, making it difficult to separate the individual effect of each variable on the model.

Mathematical definition: For each feature X_i , the VIF is computed via a linear regression where X_i is used as the dependent variable and all the remaining features as independent variables. The formula is:

$$\text{VIF}_i = \frac{1}{1 - R_i^2}$$

where R_i^2 is the coefficient of determination of the regression of X_i on the other variables.

Interpretation:

- $\text{VIF} = 1$: No correlation.
- $1 < \text{VIF} \leq 5$: Moderate correlation (acceptable).
- $5 < \text{VIF} \leq 10$: Moderate to high multicollinearity, further investigation is required.
- $\text{VIF} > 10$: High multicollinearity (removal of the variable is required).

A.2 The ReduceLROnPlateau mechanism

The ReduceLROnPlateau technique is a callback mechanism in Keras/TensorFlow that dynamically adjusts the learning rate during training.

Operation: The mechanism monitors a specific metric, in our case the `val_f1_score`. When this metric stops improving for a specified number of epochs (defined as “patience”), the algorithm reduces the learning rate by multiplying it by a reduction factor (e.g. `factor=0.5`).

Rationale: In deep neural networks, a high learning rate is useful at the beginning for a fast descent towards the minimum error. However, as the model approaches the global minimum, a high rate can cause oscillations around it. Reducing the LR allows the model to take smaller, more accurate steps (“fine-tuning”), achieving better convergence and a lower final error.

A.3 The CatBoost algorithm (Categorical Boosting)

CatBoost is a machine learning algorithm belonging to the Gradient Boosting on Decision Trees (GBDT) family. It was developed by Yandex and stands out for its ability to efficiently han-

de tabular data containing categorical variables, solving structural problems faced by other algorithms such as XGBoost or LightGBM.

Its superiority is based on three main architectural innovations:

1. Handling categorical data (Ordered Target Statistics). The traditional conversion of categorical variables to numeric ones (e.g. Target Encoding) often leads to information leakage (target leakage), since the mean value of the target is computed including the sample being predicted. CatBoost solves this problem by introducing the concept of artificial time. During training, the data are randomly shuffled (random permutation). For each sample, the algorithm computes the statistics of the category based exclusively on the samples that precede it in that particular random ordering. This procedure mimics the flow of time and eliminates the overfitting caused by “knowledge of the future”.

2. Addressing prediction shift (Ordered Boosting). In classical GBDT algorithms, the same data are used both for estimating the gradient and for constructing the model, leading to biased estimates (biased gradients) and the phenomenon of Prediction Shift. CatBoost applies the Ordered Boosting technique. It maintains multiple random permutations of the data and trains different models on increasing subsets. Thus, the residual for each sample is computed by a model that has not been trained on that specific sample, ensuring an unbiased estimate of the error.

3. Symmetric trees (Oblivious Trees). Unlike ordinary decision trees that grow asymmetrically, CatBoost constructs Symmetric Trees (Oblivious Trees). In this structure, the same split condition is applied across the entire depth level of the tree. This architecture offers two advantages:

- Regularisation: The structure is less prone to overfitting as it is simpler and more balanced.
- Execution speed: The symmetry allows extremely fast execution during the prediction phase (inference), as the decision path can be implemented very efficiently at the processor level.

For more details and the mathematical documentation of the algorithm, you can refer to the original publication: <https://arxiv.org/pdf/1706.09516> (Prokhorenkova et al., 2018).

A.4 SHAP theory (SHapley Additive exPlanations)

SHAP is a method for interpreting “black box” models that is based on Cooperative Game Theory. It was introduced by Lloyd Shapley (Nobel Prize in Economics).

Mechanism of operation: Imagine that the model’s prediction is the “gain” of a game and the features are the “players” who cooperate to achieve this gain. SHAP computes the marginal contribution of each feature, examining all possible combinations (coalitions) of features.

Basic equation: The SHAP value for a feature is the weighted average of the difference that the presence of the feature causes in the prediction, compared to its absence, across all possible combinations.

Additivity property: The sum of the SHAP values of all features plus the mean prediction value (Base Value) always equals the final prediction of the model for the specific sample:

$$f(x) = \phi_0 + \sum_{i=1}^M \phi_i$$

In our work, the TreeExplainer was used, an optimised variant of the algorithm specifically for tree-based models (such as CatBoost), which allows the exact computation of SHAP values with low computational cost.

B Code

The code of the assignment is provided alongside this report.